

HW1 Report — MPI Distributed SpMV (CSR)

Unity ID: fclin1

3a. Write-up

Task 1 — Allgather baseline (`spmv_mpi_allgather.c`)

The harness gives each rank a contiguous block of rows of A (1-D row-block partition) and the matching slice of x . Because `col_idx` holds *global* column indices, every rank must see the full x vector before it can compute its rows. The simplest correct communication is therefore a single `MPI_Allgatherv` at the start of `spmv_mpi()`.

`x_full` is allocated once in `spmv_mpi_setup()` and reused every iteration. The local CSR kernel then indexes directly into `x_full` with each nonzero's global column index.

Task 2 — Sparse exchange (`spmv_mpi_sparse.c`)

Instead of fetching all of x , each rank fetches only the columns its local rows actually reference that are *not* in its own x slice.

Setup (built once in `spmv_mpi_setup()`):

1. Scan `col_idx`; mark every remote column needed (`needed[]` bitmap).
2. Count how many entries are needed *from* each owning rank $\rightarrow$ `recv_count[]`.
3. Exchange counts with `MPI_Alltoall` so every rank learns `send_count[]`, how many of *its* entries others want.
4. Exchange the requested global column indices with `MPI_Alltoallv` so each rank learns exactly *which* of its local x entries to pack.
5. Translate received global indices into local x offsets for fast packing.

Per-iteration (`spmv_mpi()`):

- Pack the requested entries into `send_buf`, then launch non-blocking `MPI_Irecv` / `MPI_Isend` pairs and `MPI_Waitall`.
- Copy own slice into `x_buf`, then scatter the received `recv_buf` entries into `x_buf` at their global indices.
- Run the same local CSR kernel, indexing `x_buf` with global `col_idx`.

Using a full-length `x_buf` (size `num_cols`) keeps the compute loop identical to Task 1 at the cost of $O(\text{num_cols})$ memory, a trade made for simplicity and predictable access patterns.

Design decisions

- **Non-blocking sends/receives** — `MPI_Irecv`/`MPI_Isend` + `MPI_Waitall` avoids potential deadlock when every rank simultaneously posts to the same destination, and allows the MPI implementation to pipeline messages.

- **Plan built exactly once** — `setup()` is called once before the timed loop; `spmv_mpi()` never rebuilds any data structure.

Per-matrix behavior

Matrix	Key characteristic	Notable result
bfly	Fixed degree-4, perfectly balanced (imbalance = 1.000)	Best efficiency; all rows have identical nnz so every rank does equal work
Ga3As3H12, pkustk14	Dense FEM/structural (~97 nnz/row)	Highest GFLOP/s; computation dominates communication
D6-6	Rectangular; only 23,740 columns	Sparse exchange saves ~79% bytes; small $\times$ limits allgather cost
dictionary28	Word-co-occurrence; high row-length variance (imbalance up to 2.89)	Lower efficiency at high p because the heaviest rows bound the critical path
roadNet-CA	2M $\times$ 2M, very sparse (2.8 nnz/row avg.)	Poor per-rank GFLOP/s (computation trivial); sparse exchange saves >99% bytes due to local graph structure

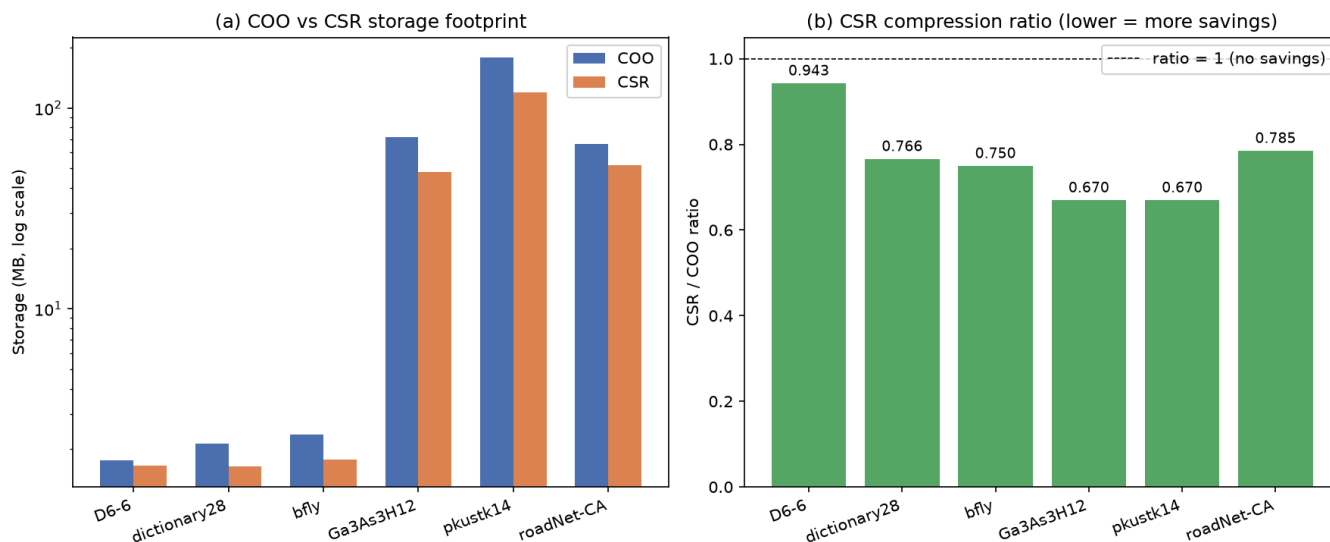
Anomalies

- **bfly** shows near-linear speedup to p=8 but flattens at p=16 under allgather: 49,152 rows / 16 = 3,072 rows per rank, and the allgather must still move all 49,152 $\times$ entries. The sparse exchange dramatically recovers efficiency here.
- **roadNet-CA allgather** at p=8 sends 55 MB per rank per iteration — over 200 $\times$ more than the sparse exchange (138 KB total) — yet setup time for allgather is negligible (<0.02 ms) while sparse setup costs ~9 ms. The crossover in per-iteration cost justifies sparse exchange for repeated SpMV.

3b. Format Storage (CSR vs COO)

Measured results

Matrix	m	nnz	nnz/row	COO (MB)	CSR (MB)	CSR/COO
D6-6	120,576	146,880	1.22	1.76	1.66	0.940
dictionary28	52,652	178,076	3.38	2.14	1.64	0.765
bfly	49,152	196,608	4.00	2.36	1.77	0.750
Ga3As3H12	61,349	5,970,947	97.3	71.65	48.01	0.670
pkustk14	151,926	14,836,504	97.7	178.04	119.30	0.670
roadNet-CA	1,971,281	5,533,214	2.81	66.40	52.15	0.785



Why the ratio varies: The ratio varies due to the difference in the number of nonzeros per row. Matrices with barely more than one nonzero per row save almost nothing due to the change in format, while matrices with multiple non zeros per row save considerably.

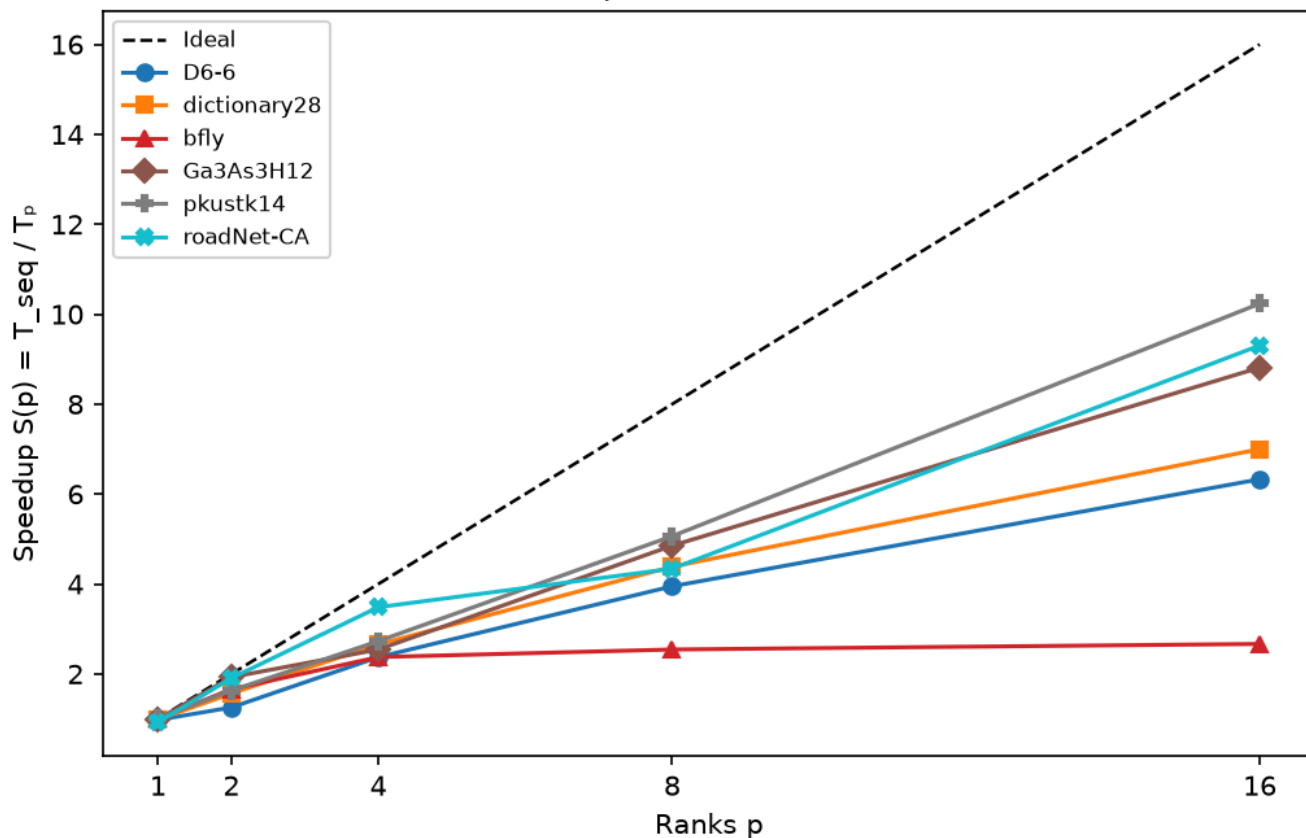
3c. Performance & Scaling

T1 is the **sequential CSR baseline** time (from `spmv_csr`, single process, no MPI), as measured by the harness before any parallel run. Using the sequential baseline as **T1** gives the true speedup of the parallel code relative to the best serial implementation, rather than hiding MPI startup overhead inside **T1**.

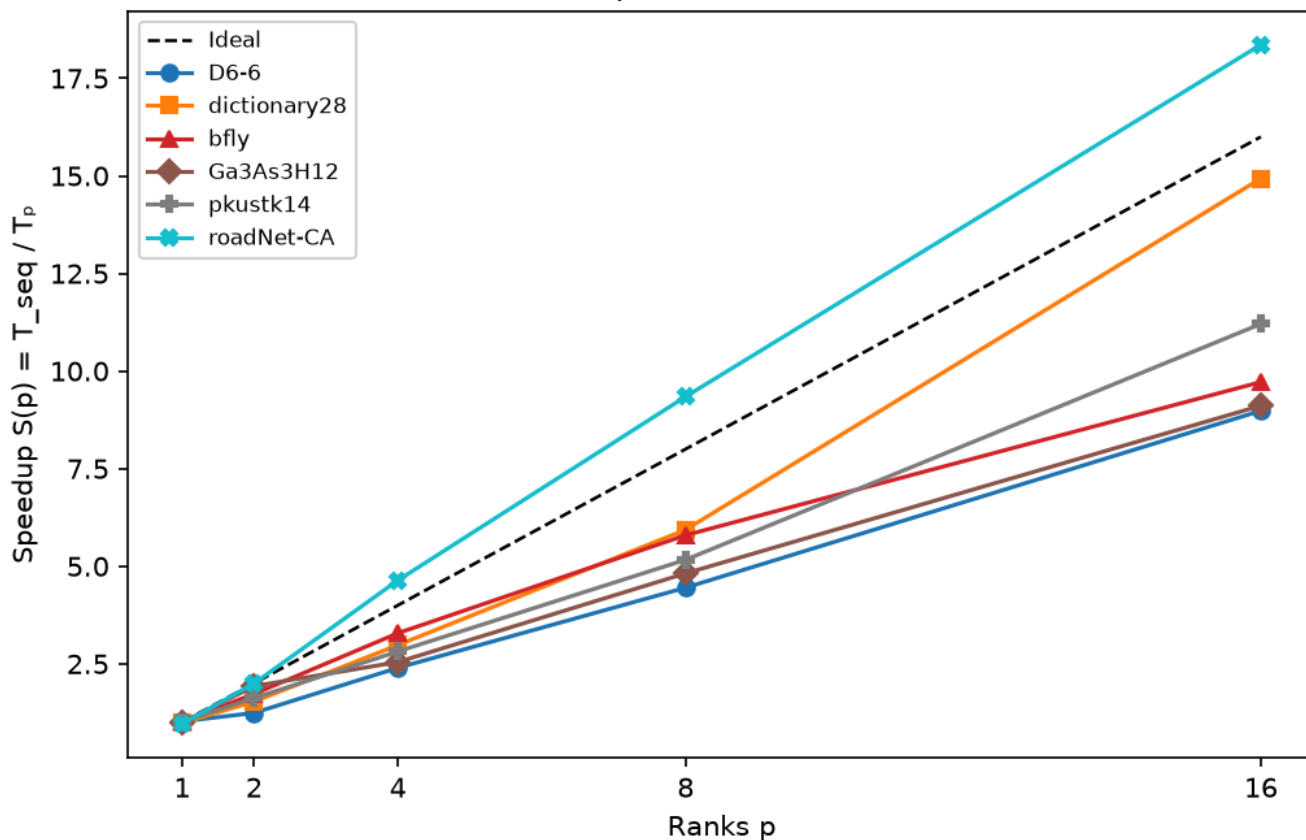
Sequential baseline times: D6-6 0.491 ms, dictionary28 0.587 ms, bfly 0.203 ms, Ga3As3H12 5.405 ms, pkustk14 12.418 ms, roadNet-CA 17.301 ms.

Speedup

Strong Scaling Speedup — Allgather
(T_1 = sequential CSR baseline)



Strong Scaling Speedup — Sparse Exchange
(T_1 = sequential CSR baseline)

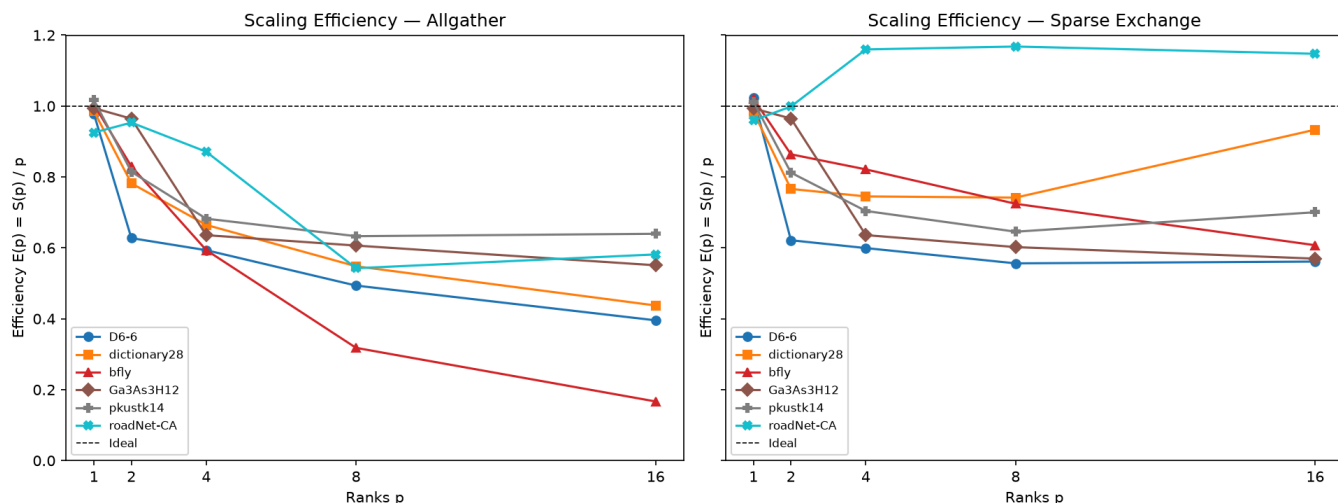


Selected speedup values (allgather), T_1 = sequential baseline:

Matrix	S(1)	S(2)	S(4)	S(8)	S(16)
D6-6	0.98	1.25	2.37	3.95	6.33
dictionary28	0.99	1.56	2.66	4.39	7.00
bfly	1.00	1.66	2.37	2.54	2.67
Ga3As3H12	0.99	1.93	2.54	4.85	8.81
pkustk14	1.02	1.63	2.73	5.06	10.23
roadNet-CA	0.93	1.91	3.48	4.34	9.30

Note that $S(1) < 1$ for most matrices — the MPI-1-rank path has a small overhead over the pure sequential code. roadNet-CA at $p=1$ is 8% slower than sequential because the MPI-1 allgather must still allocate and zero the full `x_full` buffer. Compute-heavy matrices (Ga3As3H12, pkustk14) scale best because large nnz amortizes communication cost. bfly stalls after $p=8$ as explained in §3a.

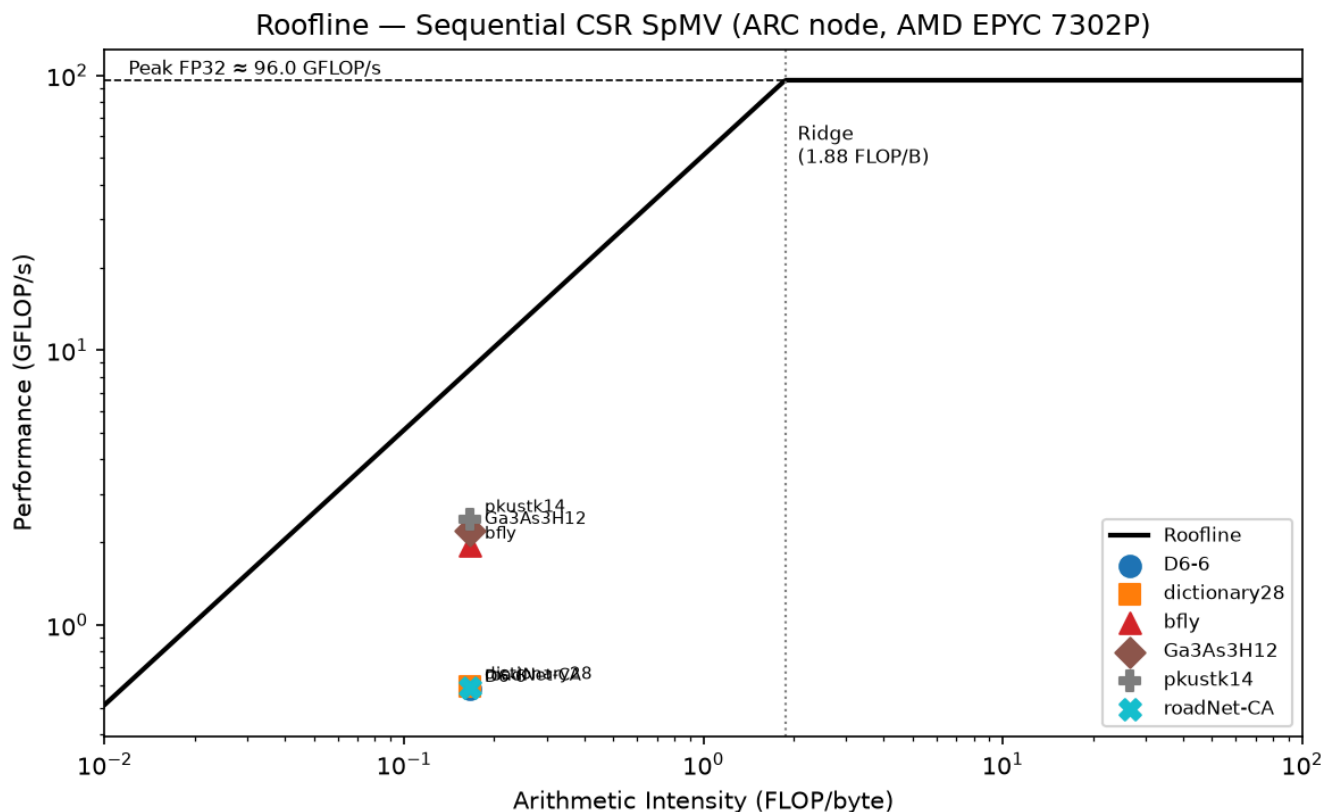
Efficiency



Sparse exchange consistently matches or exceeds allgather efficiency. Notable cases with $T1 =$ sequential baseline:

- **bfly** $E(16)$: allgather 0.17 $\rightarrow$ sparse exchange **0.61** — the sparse plan eliminates most of the allgather's redundant traffic, recovering much of the lost efficiency.
- **roadNet-CA** $E(8)$: allgather **0.54** $\rightarrow$ sparse exchange **1.17** — super-linear speedup occurs because each rank's working set (its row block + a tiny slice of remote `x`) fits in cache, whereas the sequential code must stream the entire 2 M-element `x` through DRAM. This is a cache-size effect, not a measurement error.

Roofline



SpMV arithmetic intensity at 12 bytes/nonzero (4 B `vals` + 4 B `col_idx` + 4 B `x[col]`) and 2 FLOPs/nonzero:

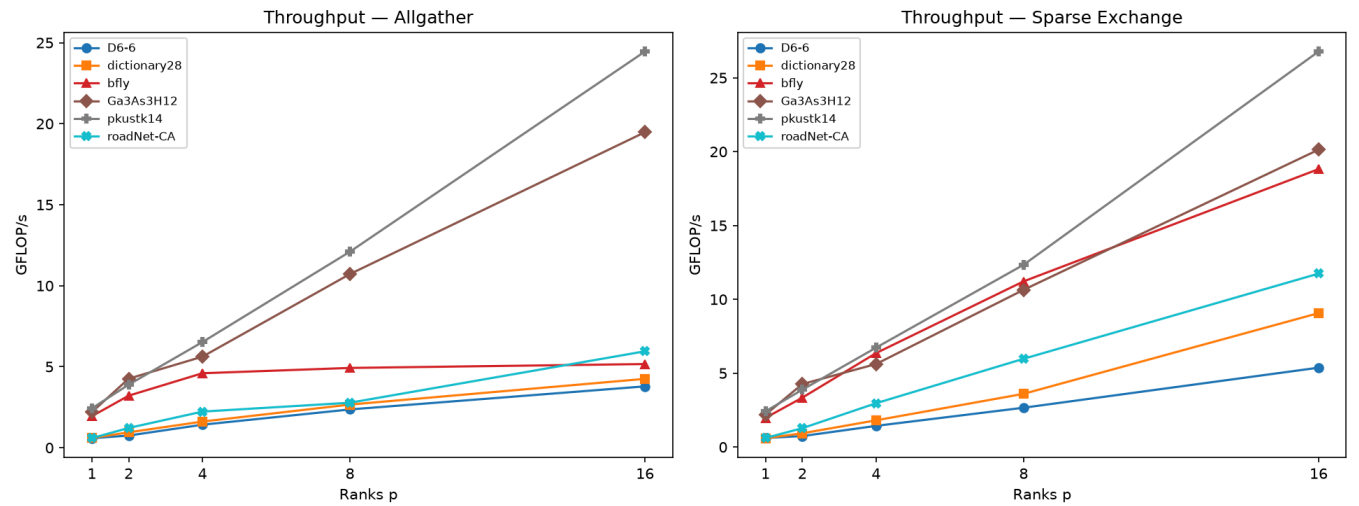
$$I = \frac{2}{12} \approx 0.167 \text{ FLOP/byte}$$

The memory-bandwidth ceiling at this intensity: $0.167 \times 51.2 \text{ GB/s} \approx 8.6 \text{ GFLOP/s}$.

All matrices measure 0.59–2.43 GFLOP/s at $p = 1$ — well below the 8.6 GFLOP/s memory-bound ceiling. This confirms that **SpMV is strongly memory-bound**. The further gap from 8.6 GFLOP/s is explained by:

- **Irregular access to `x`**: random `col_idx` values cause cache misses, reducing effective DRAM bandwidth below the hardware peak.
- Matrices with higher nnz/row (Ga3As3H12, pkustk14 ≈ 97 nnz/row) achieve the best GFLOP/s because consecutive nonzeros in the same row share a spatial neighborhood in `x`, improving cache reuse.
- Matrices with low nnz/row (D6-6 ≈ 1.2 , roadNet-CA ≈ 2.8) are the furthest from the memory ceiling — nearly every `x` access misses, and there is too little computation to hide latency.

GFLOP/s vs. ranks



3d. Communication Analysis

Counting model

For `MPI_Allgatherv` we use the **ring allgather model**: each rank sends one segment per step for $p - 1$ steps. The harness sums `comm_stats.bytes_sent` over all ranks; we set that field to $(p - 1) \times \text{num_cols} \times \text{sizeof(float)}$ per rank (each rank ships its local slice $p - 1$ times around the ring). Summed over all p ranks, total bytes = $p \times (p - 1) \times \text{num_cols} \times 4$.

For the sparse exchange, `comm_stats.messages` counts actual `MPI_Isend` calls (one per peer that requested entries), and `bytes_sent` counts bytes in the packed `send_buf`.

Allgather — analytical counts

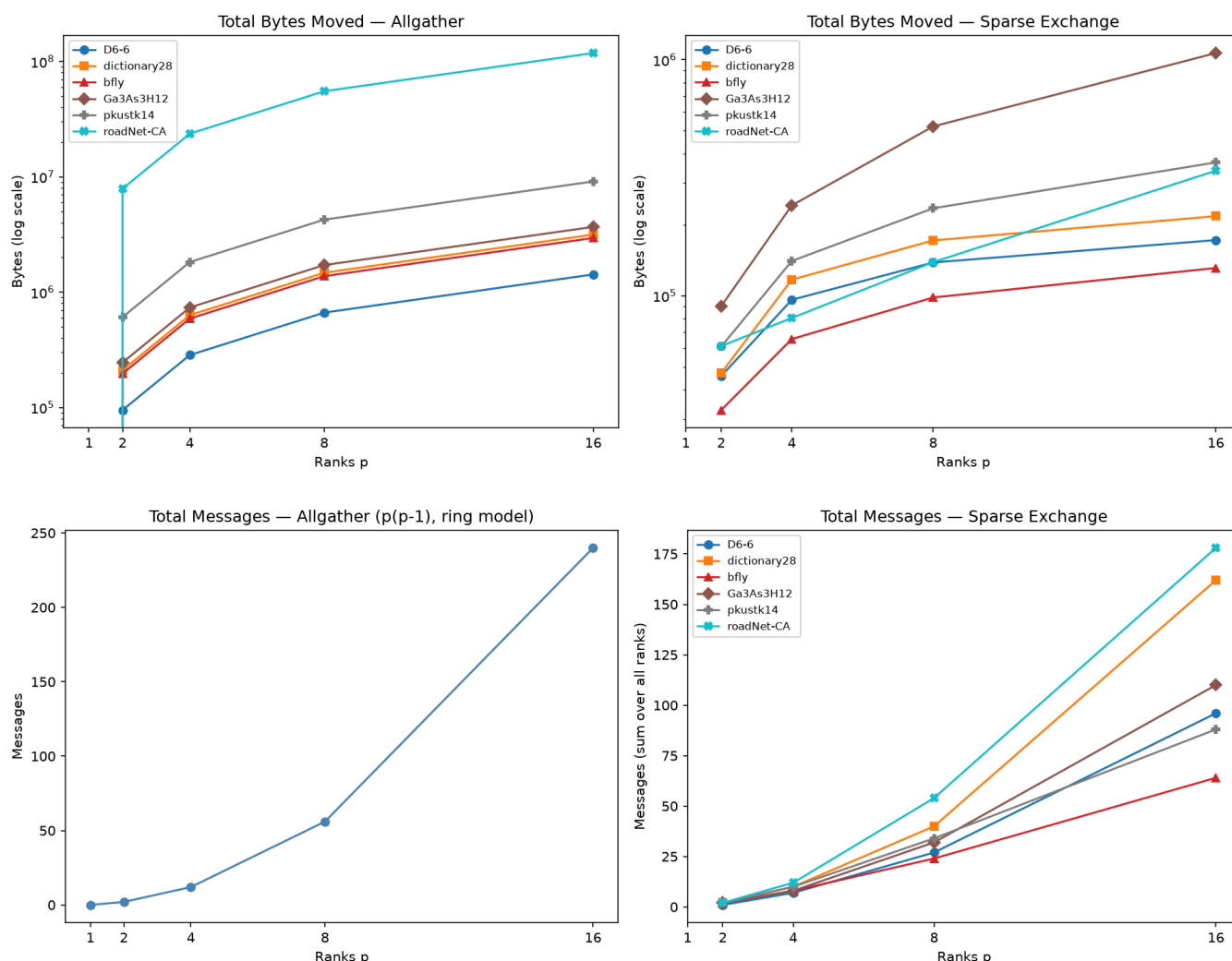
Quantity	Formula	p=8, roadNet-CA
Messages per rank	$p - 1$	7
Bytes per rank (sent)	$(p-1) \times n \times 4$	55.2 MB
Total messages	$p(p-1)$	56
Total bytes	$p(p-1) \times n \times 4$	441 MB

The CSV-reported `bytes` field equals the per-rank value $(p-1) \times n \times 4$, which matches for roadNet-CA at p=8: $7 \times 1,971,281 \times 4 = 55,195,868 \checkmark$.

Sparse exchange — measured at p = 8

Matrix	Msgs (total)	Bytes (total)	Allgather bytes	Reduction
D6-6	27	138,344	664,720	79%
dictionary28	40	171,544	1,474,256	88%
bfly	24	98,304	1,376,256	93%
Ga3As3H12	32	521,652	1,717,772	70%

Matrix	Msgs (total)	Bytes (total)	Allgather bytes	Reduction
pkustk14	34	234,724	4,253,928	94%
roadNet-CA	54	138,772	55,195,868	99.75%



Which matrices benefit most

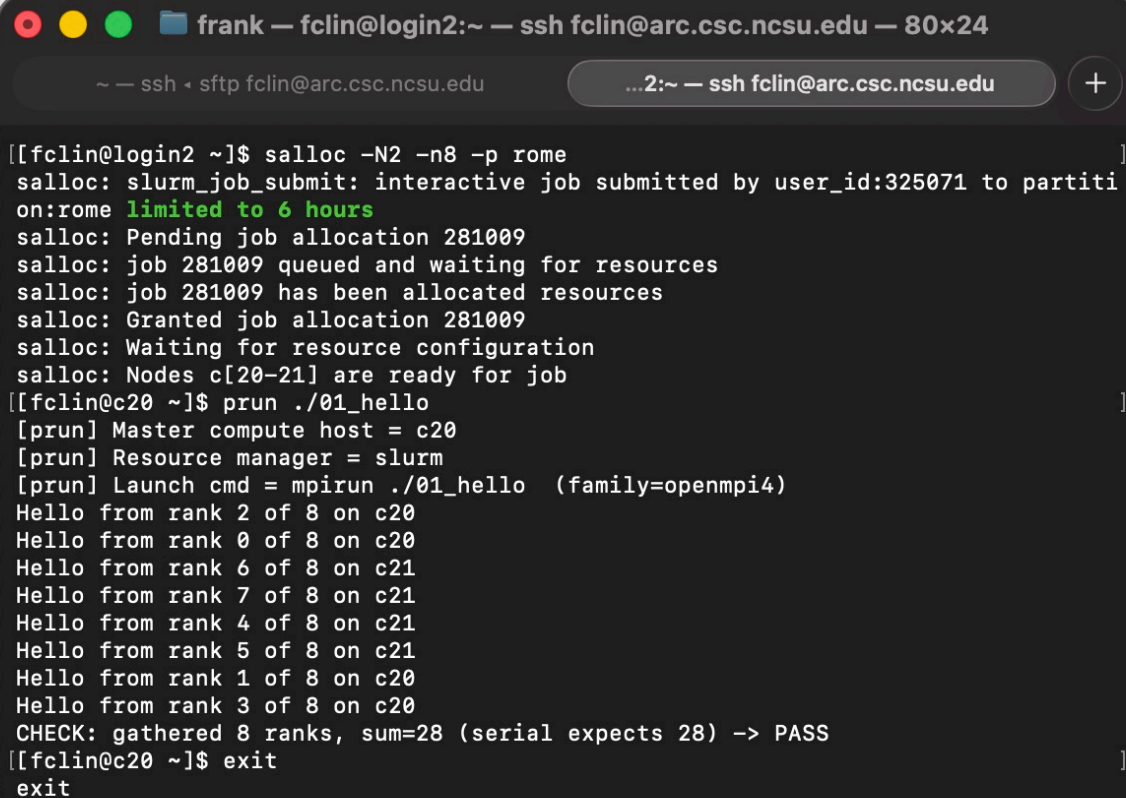
roadNet-CA benefits most by far (>99% byte reduction). Its 2 M columns make the allgather enormous, yet the road network is geographically local — each rank's row-block corresponds to a contiguous range of node IDs, and road edges connect nearby nodes. As a result, most nonzero column references fall within the rank's own block or immediately adjacent blocks, so the sparse exchange moves only a tiny fraction of x .

pkustk14 and **bfly** each save >90%. pkustk14 is a structural FEM stiffness matrix where nearby DOFs are strongly coupled; the row-block partition keeps most coupling within rank. bfly's butterfly topology means rank p 's rows connect to only two specific other ranks' columns.

Ga3As3H12 saves least (70%) because its quantum-chemistry off-diagonal entries are more scattered — a rank's rows reference columns distributed across many other ranks, so the sparse plan must fetch entries from nearly every peer.

Message count: Sparse exchange total messages grow roughly as $O(p \times \text{avg_neighbors})$ rather than the allgather's strict $p(p-1)$. For locally structured matrices (bfly, roadNet-CA), the average neighbor count stays small so sparse exchange uses fewer messages even though the allgather message count (56 at $p=8$) is a small number too. The dominant saving is in *bytes*, not messages.

Extra: ARC Access Proof



```
frank — fclin@login2:~ — ssh fclin@arc.csc.ncsu.edu — 80x24
~ — ssh — sftp fclin@arc.csc.ncsu.edu  ...2:~ — ssh fclin@arc.csc.ncsu.edu +

[[fclin@login2 ~]$ salloc -N2 -n8 -p rome ]
salloc: slurm_job_submit: interactive job submitted by user_id:325071 to partiti
on:rome limited to 6 hours
salloc: Pending job allocation 281009
salloc: job 281009 queued and waiting for resources
salloc: job 281009 has been allocated resources
salloc: Granted job allocation 281009
salloc: Waiting for resource configuration
salloc: Nodes c[20-21] are ready for job
[[fclin@c20 ~]$ prun ./01_hello ]
[prun] Master compute host = c20
[prun] Resource manager = slurm
[prun] Launch cmd = mpirun ./01_hello (family=openmpi4)
Hello from rank 2 of 8 on c20
Hello from rank 0 of 8 on c20
Hello from rank 6 of 8 on c21
Hello from rank 7 of 8 on c21
Hello from rank 4 of 8 on c21
Hello from rank 5 of 8 on c21
Hello from rank 1 of 8 on c20
Hello from rank 3 of 8 on c20
CHECK: gathered 8 ranks, sum=28 (serial expects 28) -> PASS
[[fclin@c20 ~]$ exit ]
exit
```